

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

GROUP 3 SC2006 - SCSD

LAB 3 DELIVERABLES

Team Members
1. Vivek Shrey (U2440609F) - Team Leader
2. Ching Xin Wei (U2421032K)
3. Dharshini Saravanan (U2420758E)
4. Li Sinuo (U2421736H)
5. Sivasankar Sivagnana Sundari (U2420528F)

I. Use Case Diagram



Actors

- **User** – a job seeker who browses, searches, manages profile/skills, uploads resume, saves jobs, logs applications, and views recommendations/trends.

Potential Narrative for a user:

A **User** can **Register** (UC-1) then **Login** (UC-2). Once authenticated, they can **Manage Profile & Skills** (UC-3) to improve recommendations. They can explore **Sector Trends** (UC-5) or directly **View Recommended Jobs** (UC-6) and **Search & Filter Jobs** (UC-7). Both lead to **View Job Details** (UC-8). From **Job Details**, the user may **Save/Unsave** (UC-9), **Log/Update Application** (UC-10), or **Fetch Courses for Matching Skill Gaps**

(UC-11). This way, the user helps themselves find a job - inclining to their interests.

Use Cases

1. **UC-1 Register**
2. **UC-2 Login**
3. **UC-3 Manage Profile & Skills**
4. **UC-4 Upload Resume**
5. **UC-5 View Sector Trends**
6. **UC-6 View Recommended Jobs**
7. **UC-7 Search & Filter Jobs**
8. **UC-8 View Job Details**
9. **UC-9 Save/Unsave Job**
10. **UC-10 Log/Update Application**
11. **UC-11 Fetch Courses for Matching Skill Gaps**

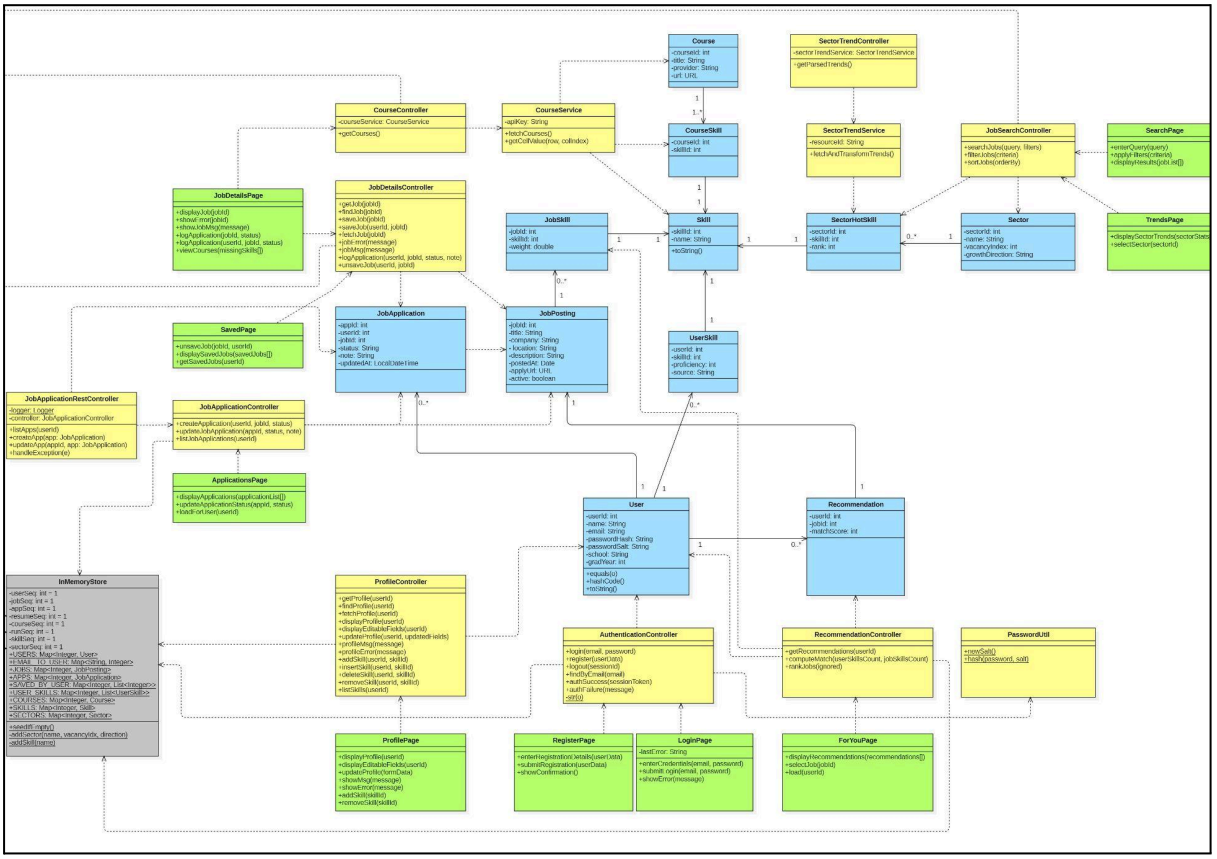
Include / Extend Relationships

- **Includes**
    - **UC-1 Register** is «include» in **UC-3/UC-4/UC-5/UC-6/UC-7** (first-time users reach those via registration).
    - **UC-2 Login** is «include» in **UC-1 Register** (auto-login after registering).
    - **UC-8 View Job Details** is «include» in **UC-6 View Recommended Jobs** and **UC-7 Search & Filter Jobs** (both drill into details).
- **Extends**
    - **UC-9 Save/Unsave Job** «extend» **UC-8 View Job Details** (optional bookmarking from the details page).
    - **UC-10 Log/Update Application** «extend» **UC-8 View Job Details** (optional apply/log action from details).
    - **UC-11 Fetch Courses for Matching Skill Gaps** «extend» **UC-8 View Job Details** (optional upskilling suggestions from details).

II. Design Model

A. Class Diagram

If the image is unclear, please refer to the separate file on GitHub.



User Interfaces (Boundary Classes)

- **SearchPage** – enter a query, apply filters, display paged results; uses `JobSearchController`.
- **TrendsPage** – browse sector trends and vacancy index; uses `JobSearchController`.
- **JobDetailsPage** – show job info, save/unsave, apply; uses `JobDetailsController`.
- **SavedPage** – list saved jobs; uses `ApplicationController`.
- **ApplicationsPage** – list and update application statuses; uses `JobApplicationController`.
- **ProfilePage** – view/edit profile, skills, resume upload; uses `ProfileController`.
- **ForYouPage** – show ranked recommendations; uses `RecommendationController`.
- **LoginPage / RegisterPage** – auth flows; use `AuthenticationController`.

Controllers (Control Classes)

- **JobSearchController**
  `searchJobs(query, filters)`; `sortJobs(orderBy)`; fetches jobs by sector/keywords and vacancy index; consults `Sector`, `JobPosting`.
- **JobDetailsController**
  `getJob(jobId)`; `saveJob(userId, jobId)`; `unsaveJob(...)`; `applyToJob(...)`; logs message and status updates; touches `JobPosting`, `Application`, `User`.
- **JobApplicationController**
  `createJobApplication(userId, jobId, status)`; `updateJobApplication(appId, status, note)`; `listJobApplications(userId)`; manages lifecycle of `Application`.
- **ProfileController**
  `getProfile(userId)`; `updateProfile(...)`; `addSkill(...)`; `removeSkill(...)`; manages `User`, `UserSkill`.

- **AuthenticationController**
  `login(email, password)`; `register(userData)`; `logout(sessionId)`; returns/invalidates `sessionId` and errors; accesses `User`.

- **RecommendationController**
  `getRecommendations(userId)`; `computeMatch(resume, jobSkills)`; `rankJobs(scores)`; writes `Recommendation(userId, jobId)`.

- **CourseController**
  `getCourses()`; maps `JobSkill` gaps to `Course` via `CourseSkill`.

App Object Layer (Entity Classes)

- **User** (`userId`, `name`, `email`, `passwordHash`, `passwordSalt`, `school`, `gradYear`) – has many `UserSkill`, `Application`, `Recommendation`.

- **Skill** (`skillId`, `name`) – linked to `User` via `UserSkill` and to `JobPosting` via `JobSkill`.

- **UserSkill** (`userId`, `skillId`, `proficiency`, `source`).

- **JobPosting** (`jobId`, `title`, `company`, `location`, `description`, `postedAt`, `applyUrl`, `active`) – has many `Application` and `JobSkill`.

- **JobSkill** (`jobId`, `skillId`, `weight`) – required skills per job.

- **Sector** (`sectorId`, `name`, `vacancyIndex`, `growthDirection`) – ranked by **SectorHotSkill**.

- **SectorHotSkill** (`sectorId`, `skillId`, `rank`) – hotness ranking inside a sector.

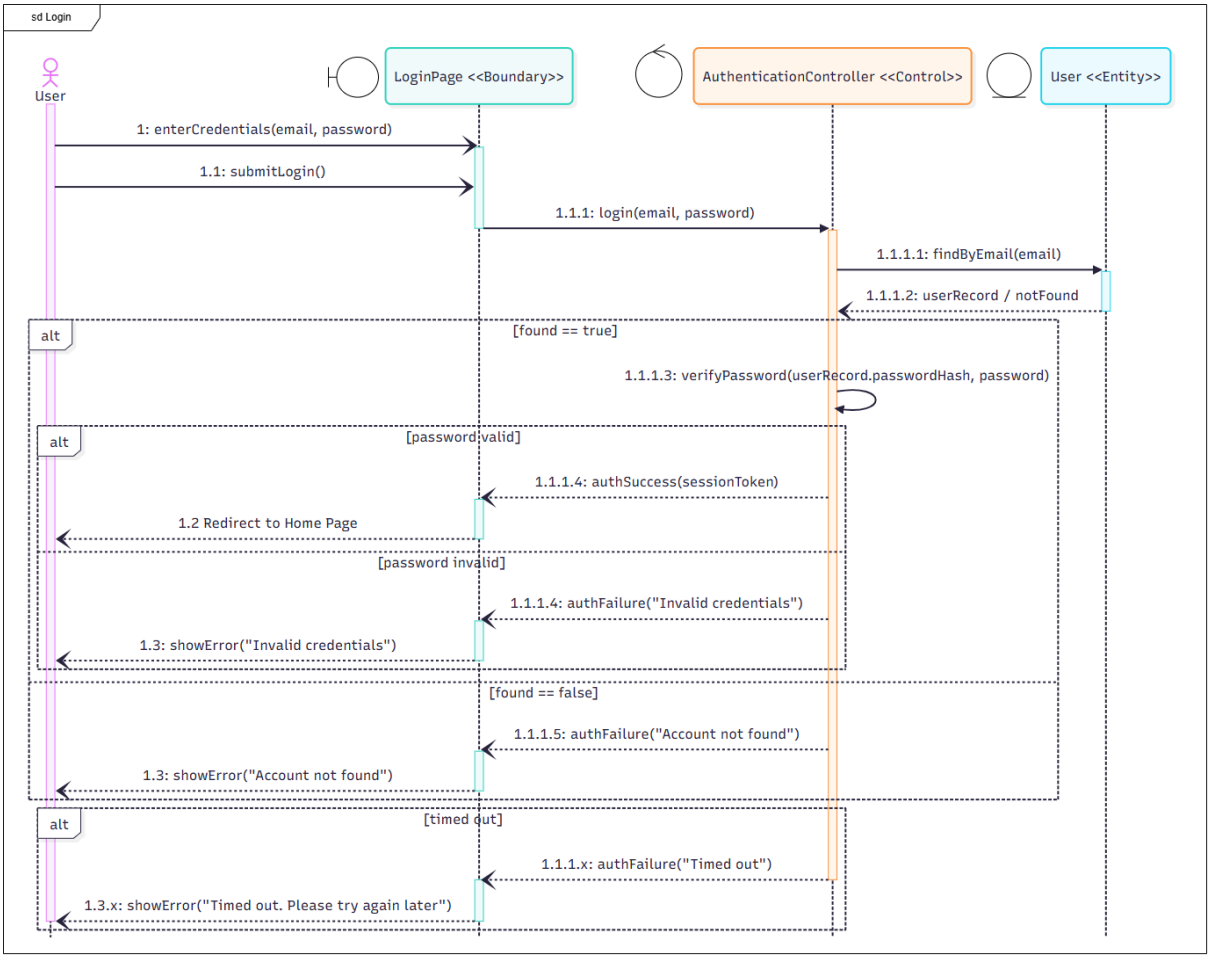
- **JobApplication** (`appId`, `userId`, `jobId`, `status`, `note`, `updatedAt`).

- **Recommendation** (`userId`, `jobId`) – a 1..* for user to ranked jobs.

- **Course** (`courseId`, `title`, `provider`, `url`) – mapped to skills via **CourseSkill**.
- **CourseSkill** (`courseId`, `skillId`) – what a course teaches.

B. Sequence Diagrams

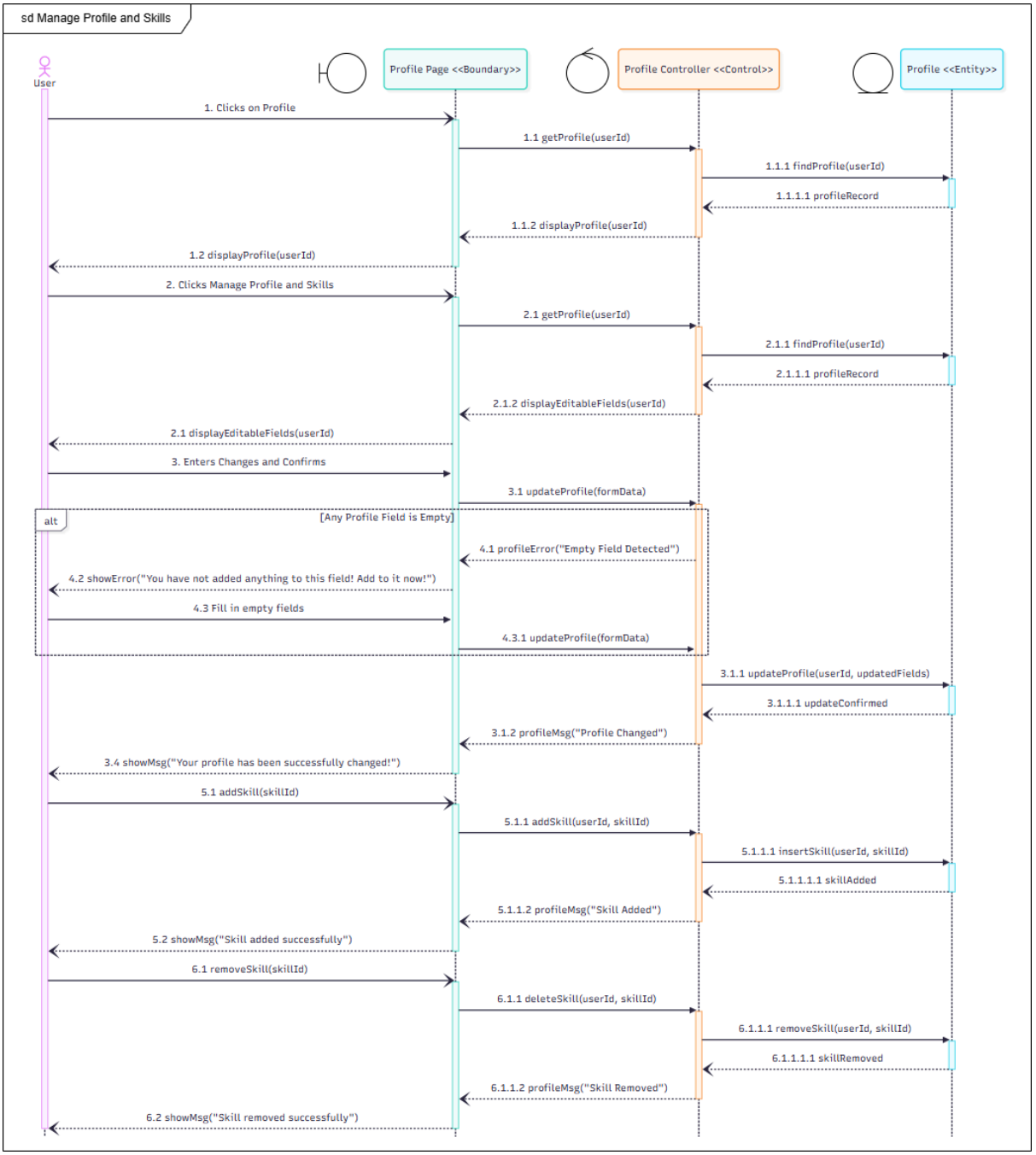
Use Case - UC-2 (Login)



Description:

Users log in to the system by inputting their email and password. If their account email is found in the system records, the system will check their password to see if it is correct. If correct, the user is redirected to the Home Page. Otherwise, the system informs them that the password is incorrect. If the email is not found in the records, the system informs them that the account does not exist. If the system times out and fails the authentication of the email and password, the system sends an error message and informs the user.

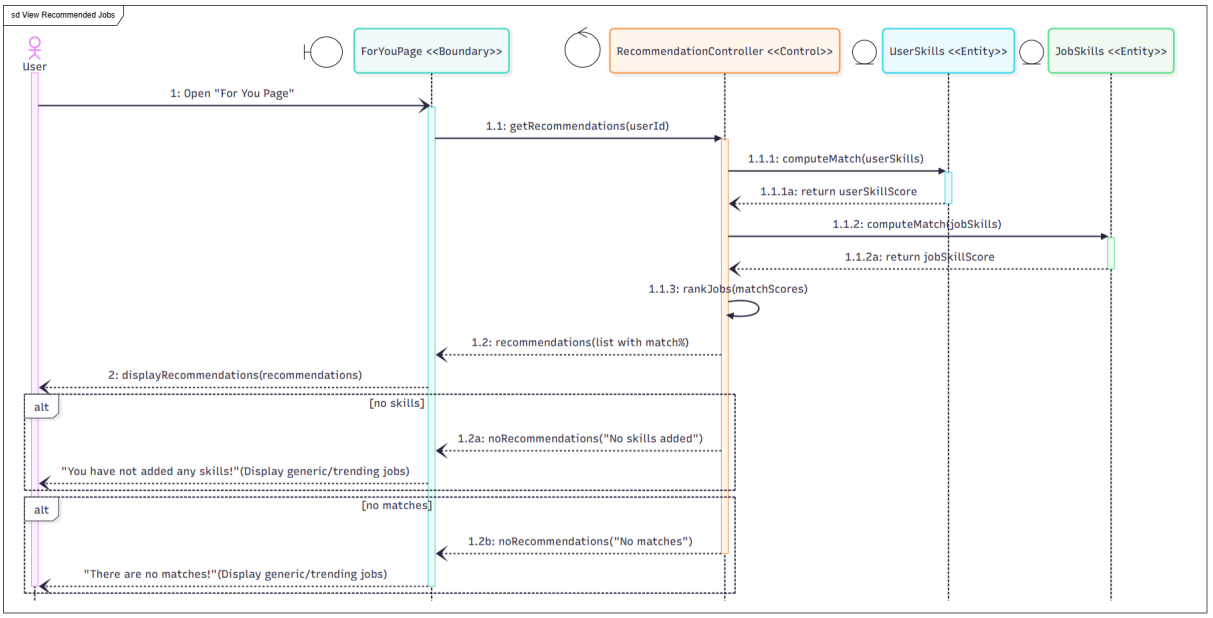
Use Case - UC-3 (Manage Profile and Skills)



Description:

The user clicks on the profile page to view it. On the page, there is an option to manage profile and skills which allows them to adjust the profile fields. After they input their changes, they can submit them to the system. The system checks the profile fields and confirms their updates if they are filled up. Otherwise if it detects an empty field, it prompts the user to fill them up. After they have been filled up, the system updates and confirms the changes, informing the user the changes to their profile have been made successfully. The user can also add and delete skills from their profile.

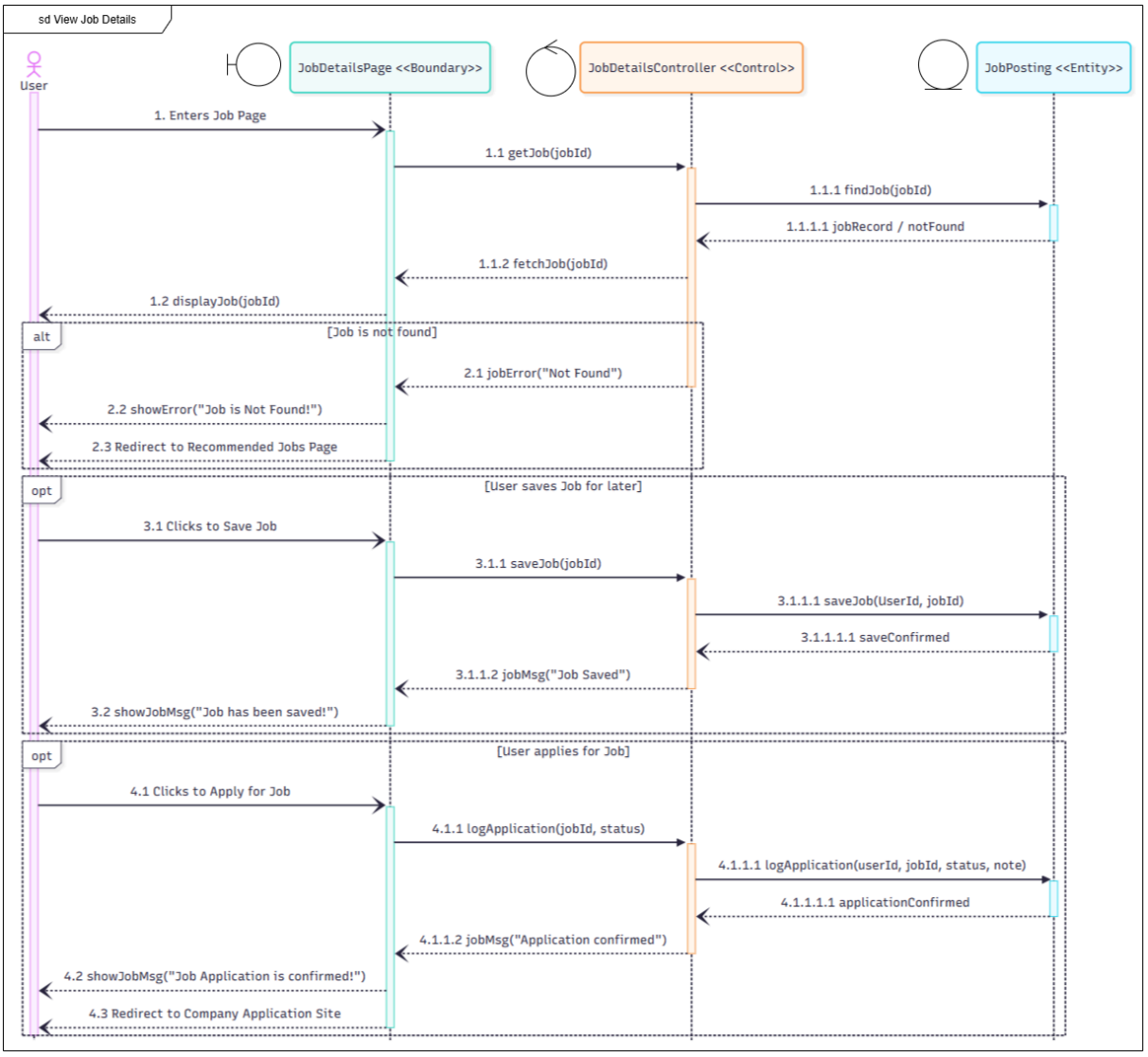
Use Case - UC-6 (View Recommended Jobs)



Description:

User clicks on the For You page, which shows recommended jobs to the user based on the skills in their profile. The system retrieves the jobs by matching the user skills to the job requirements and displaying them to the user on the page when a match is found. If the user has no skills in their profile or there are no matches which brings up no recommendations, the user is informed that they have no skills added and the system instead displays generic or trending jobs.

Use Case - UC-8 (View Job Details)

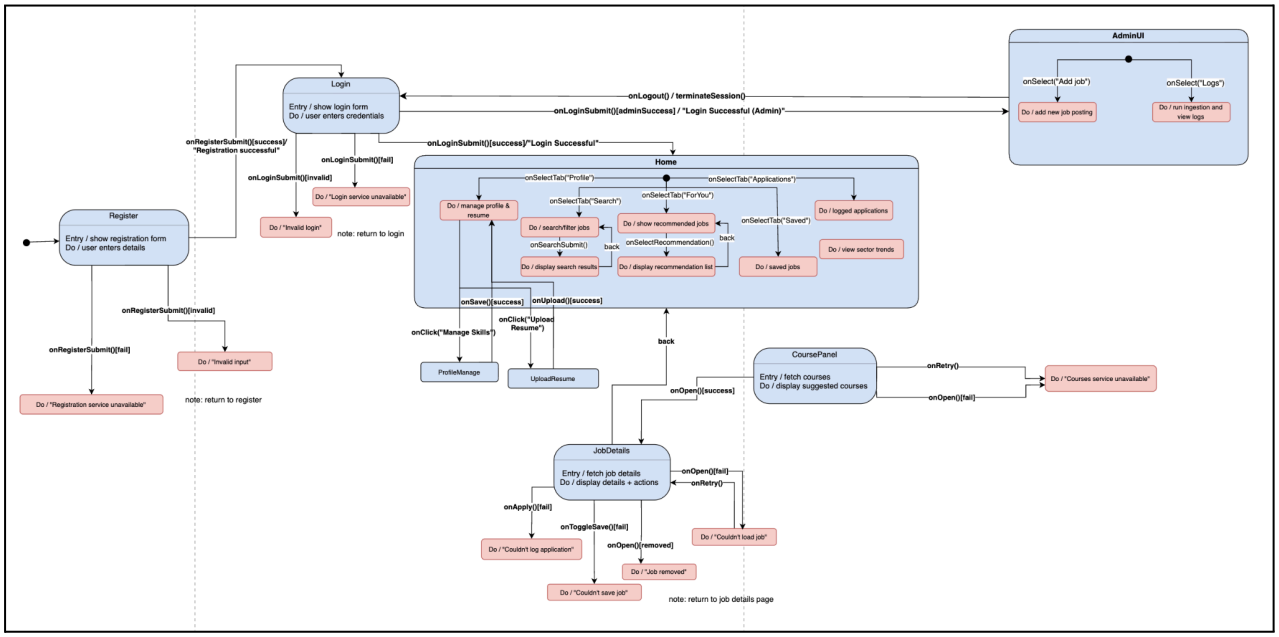


Description:

The user enters the Job page, which displays the details of the job that the system has retrieved for the user. If the job is found, it is displayed to the user on the page. Otherwise, the system informs the user the job is not found and redirects them to the Recommended Jobs page. The user has the option to save the job for later when they want to have another look at it another time. They also have the option to apply for the job which logs their application in the system, recording and informing the user that they have applied for the job. After which, they are redirected towards the company's application site.

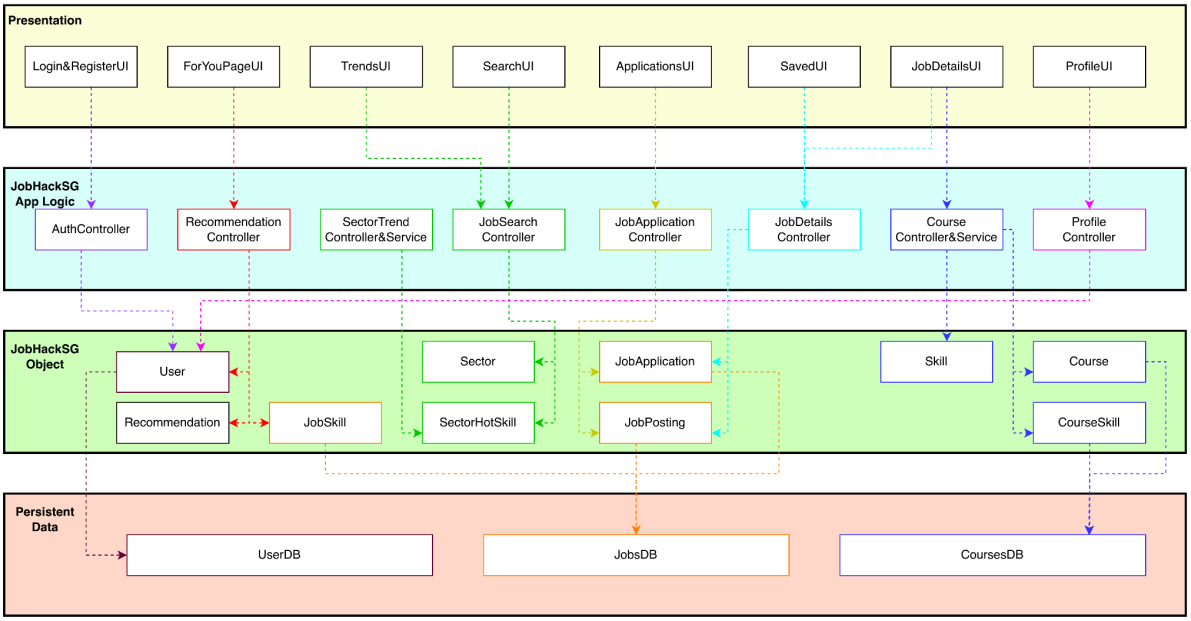
C. Dialog Map Diagram

If the image is unclear, please refer to the separate file on GitHub.



III. System Architecture

If the image is unclear, please refer to the separate file on GitHub.



1. **Presentation:**

This layer handles the interaction between the users and the application interface. It involves the UI components of the app that the user interacts with.

UI Components:

- `Login&RegisterUI`: Manages user login and registration.
- `ForYouPageUI`: Displays personalized job recommendations.
- `TrendsUI`: Shows trending sectors and skills.
- `SearchUI`: Allows users to search for job postings.
- `ApplicationsUI`: Displays the applications made by the user.
- `SavedUI`: Displays saved jobs for easy access.
- `JobDetailsUI`: Shows detailed information about the selected job.
- `ProfileUI`: Displays and manages user profile information.

2. JobHackSG App Logic Layer:

This layer contains the logic that controls the functionality and flow of the application. Controllers manage the requests between the UI and the backend services.

Key Controllers:

- `AuthController`: Handles user authentication.
- `Recommendation Controller`: Helps in fetching job recommendations based on user profile and skills.
- `SectorTrend Controller&Service`: Fetches and processes sector trends.
- `JobSearch Controller`: Handles job search functionality.
- `JobApplication Controller`: Manages job application processes.
- `JobDetails Controller`: Retrieves detailed information for job postings.
- `Course Controller&Service`: Helps in providing recommendations for courses based on skill gaps.
- `Profile Controller`: Manages user profile and skill updates.

3. JobHackSG Object Layer

This layer includes the entities and objects that interact with the database and represent real-world objects in the application. *Note: Not all entity classes from the class diagram are included here, because this is mainly representing the system level architecture.*

Main Entities:

- `User`: Represents the user of the system, including their details.
- `JobApplication`: Represents a job application submitted by a user.
- `Sector`: Represents job sectors and trends.
- `Skill`: Represents the skills relevant to users and job postings.
- `Course`: Represents available courses to fill skill gaps.
- `JobPosting`: Represents job listings that are available for application.
- `Recommendation`: Represents personalized job recommendations.

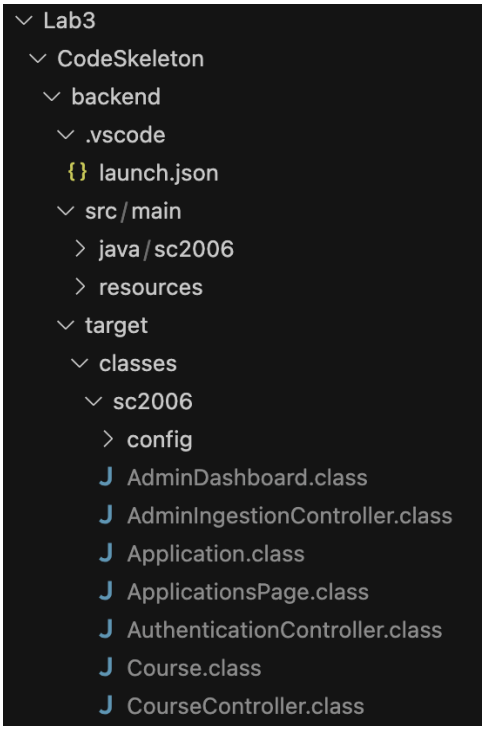
4. Persistent Data Layer

This layer handles the data storage and retrieval from the database. It includes the databases that store the system's data.

Databases:

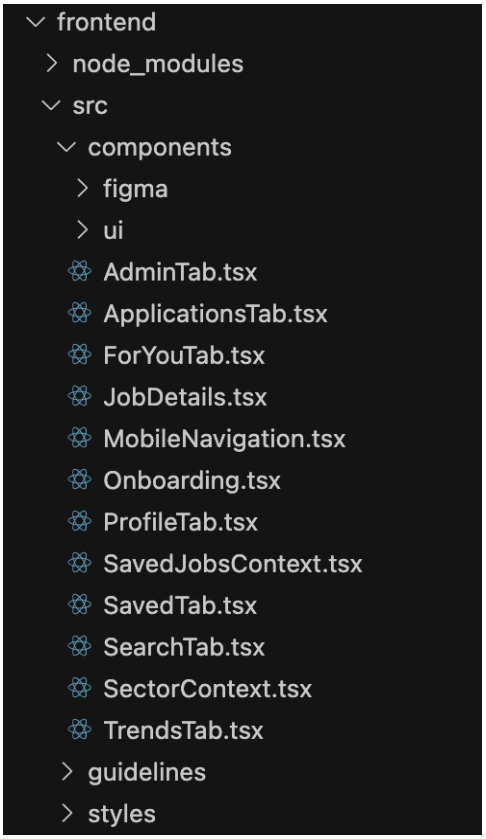
- `UserDB`: Stores user data.
- `CoursesDB`: Stores courses available for users to improve their skills.
- `JobsDB`: Contains a list of jobs posted by companies.

IV. Application Skeleton



Backend:

The backend is developed using Java Spring Boot and follows a layered architecture consisting of controllers, services, repositories, and models. Each layer has a distinct role to ensure modularity, maintainability, and clarity in data flow. The controllers define REST API endpoints (e.g., /courses, /jobs, /applications) and handle incoming HTTP requests, returning responses in JSON format. The services implement the main business logic, handling data processing, validation, and interaction between the controllers and repositories. The repositories manage communication with the database, typically by extending Spring Data JPA interfaces for CRUD operations. The models define entity classes such as Course, Job, User, and Application, which represent the structure of database tables. The backend's main entry point, Application.java, is annotated with @SpringBootApplication and is responsible for bootstrapping the application and exposing the REST endpoints.



Frontend:

The frontend is built using React with TypeScript and structured under the frontend/src directory.
It primarily consists of multiple screens (UI components), which are responsible for rendering different user interfaces such as job listings, course information, and application management.

The App.tsx is the entry point of the frontend application. Other folders contain reusable UI elements like buttons that are shared among different screens, distinct sub-screens for different user roles or functions, contains TypeScript helper files (e.g., courses.ts, applications.ts) that define the methods for calling backend REST APIs using fetch or axios and includes utility/helper functions such as date formatters, validation methods, or constants used across multiple screens.